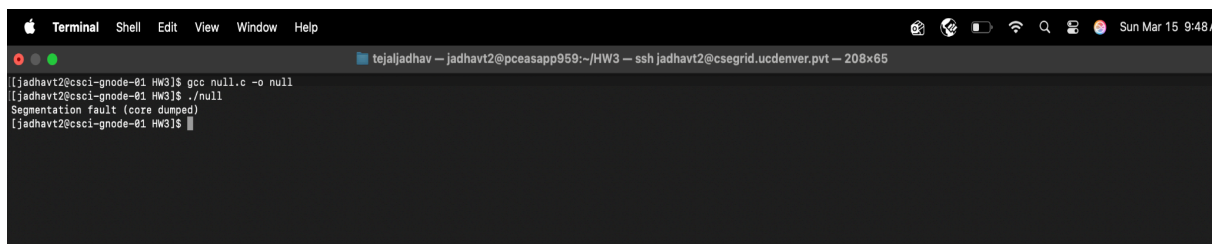


Homework Assignment 3

Questions:

1. First, write a simple program called `null.c` that creates a pointer to an integer, sets it to `NULL`, and then tries to dereference it. Compile this into an executable called `null`. What happens when you run this program?

Output:

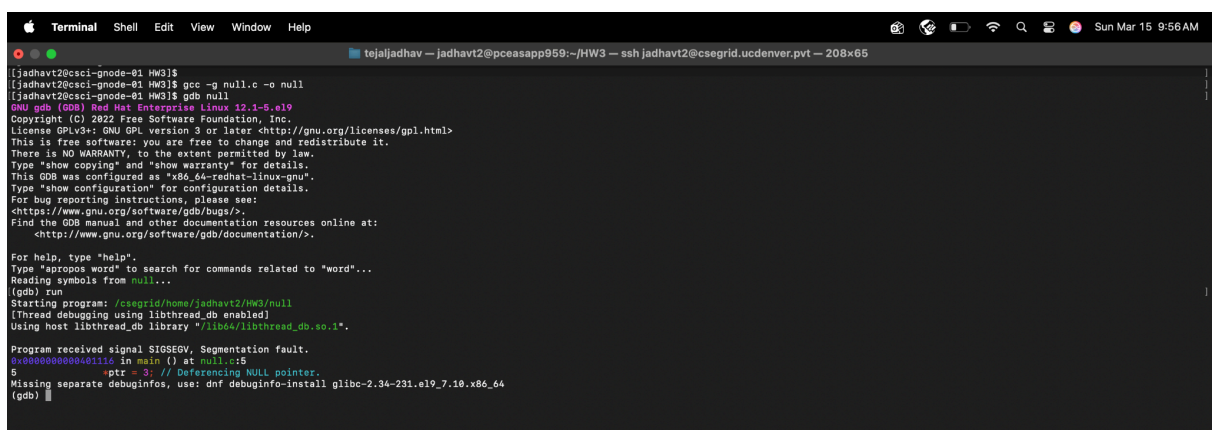


```
tejaljadhav — jadhav2@pceasapp959:~/HW3 — ssh jadhav2@csegrid.ucdenver.pvt — 208x65
[jadhav2@csci-gnode-01 HW3]$ gcc null.c -o null
[jadhav2@csci-gnode-01 HW3]$ ./null
Segmentation fault (core dumped)
[jadhav2@csci-gnode-01 HW3]$
```

When the program was run, the output displayed: Segmentation fault (core dumped)
This occurs because the pointer is set to `NULL`, meaning it does not point to any valid memory location. When the program attempts to dereference the pointer (`*ptr = 3`), it tries to access invalid memory. The operating system detects this illegal memory access and terminates the program, resulting in a segmentation fault.

2. Next, compile this program with symbol information included (with the `-g` flag). Doing so, let's put more information into the executable, enabling the debugger to access more useful information about variable names and the like. Run the program under the debugger by typing `gdb null` and then, once `gdb` is running, typing `run`. What does `gdb` show you?

Output:



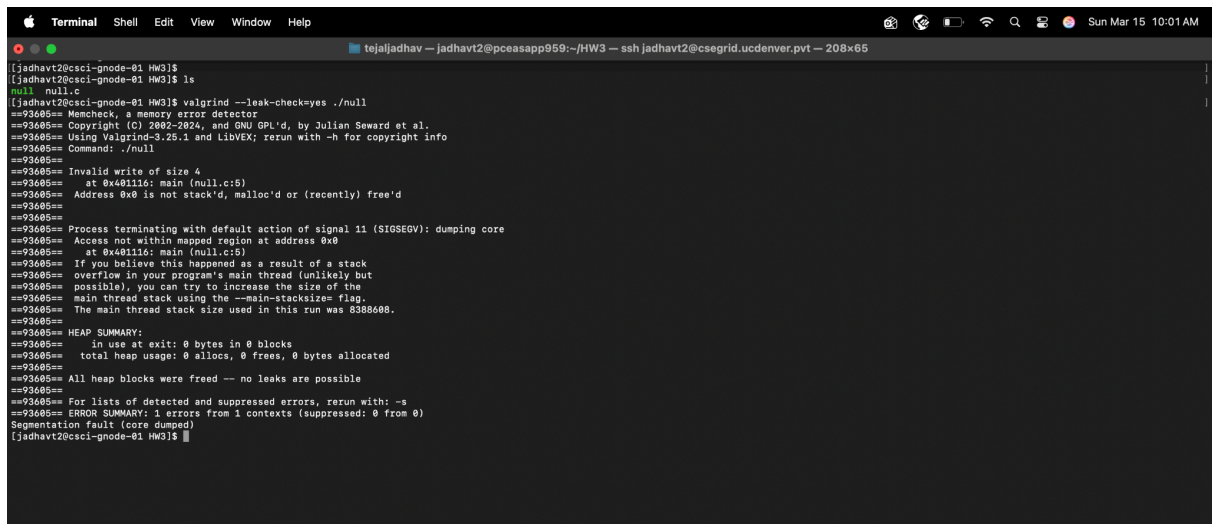
```
tejaljadhav — jadhav2@pceasapp959:~/HW3 — ssh jadhav2@csegrid.ucdenver.pvt — 208x65
[jadhav2@csci-gnode-01 HW3]$ gcc -g null.c -o null
[jadhav2@csci-gnode-01 HW3]$ gdb null
GNU gdb (GDB) Red Hat Enterprise Linux 12.1-5.el9
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from null...
(gdb) run
Starting program: /csegrid/home/jadhav2/HW3/null
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x00000000000110 in main () at null.c:5
5      *ptr = 3; // Dereferencing NULL pointer.
Missing separate debuginfos, use: dnf debuginfo-install glibc-2.34-231.el9.7.16.x86_64
(gdb)
```

When the program is run inside gdb, the debugger stops execution and displays a SIGSEGV, Segmentation fault message. It also shows the exact location of the error in the source code. The output indicates that the crash occurred in the main() function at null.c:5. The line `*ptr = 3` is highlighted as the cause of the error because the program attempts to dereference a NULL pointer. The debugger therefore helps identify the exact line where the program crashes.

3. Finally, use the valgrind tool on this program. We'll use the memcheck tool that is a part of valgrind to analyze what happens. Run this by typing in the following: `valgrind --leak-check=yes null`. What happens when you run this? Can you interpret the output from the tool?

Output:



```
tejal@tejal:~$ valgrind --leak-check=yes ./null
==3608== Memcheck, a memory error detector
==3608== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==3608== Using Valgrind-3.25.1 and libVEX; rerun with -h for copyright info
==3608== Command: ./null
==3608==
==3608== Invalid write of size 4
==3608==    at 0x401116: main (null.c:5)
==3608==    Address 0x0 is not stack'd, malloc'd or (recently) free'd
==3608==
==3608== Process terminating with default action of signal 11 (SIGSEGV): dumping core
==3608== Access not within mapped region at address 0x0
==3608==    at 0x401116: main (null.c:5)
==3608== If you believe this happened as a result of a stack
==3608== overflow in your program's main thread (unlikely but
==3608== possible), you can try to increase the size of the
==3608== main thread stack using the --main-stacksize= flag.
==3608== The main thread stack size used in this run was 8388608.
==3608==
==3608== HEAP SUMMARY:
==3608==    in use at exit: 0 bytes in 0 blocks
==3608==   total heap usage: 0 allocs, 0 frees, 0 bytes allocated
==3608==
==3608== All heap blocks were freed -- no leaks are possible
==3608==
==3608== For lists of detected and suppressed errors, rerun with: -s
==3608== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
Segmentation fault (core dumped)
tejal@tejal:~$
```

When the program runs with Valgrind, the tool reports an Invalid write of size 4 at null.c:5. It shows that the program is trying to write to memory address 0x0, which is not stack memory, heap memory, or allocated memory. This happens because the pointer is set to NULL, and the program attempts to dereference it.

Valgrind also shows that the program terminates with signal 11 (SIGSEGV), which indicates a segmentation fault caused by accessing invalid memory. The heap summary shows that no memory was allocated or freed, so there are no memory leaks, but the tool reports one memory error due to the invalid write.

4. Write a simple program that allocates memory using malloc() but forgets to free it before exiting. What happens when this program runs? Can you use gdb to find any problems with it? How about valgrind (again with the --leak-check=yes flag)?

Output:

When the program runs normally, it executes successfully and does not crash because the memory allocation itself is valid. However, the allocated memory is not released using free(), which results in a memory leak.

Running with gdb:

```
Terminal Shell Edit View Window Help
tejaljadhav — jadhav2@pcoasapp959:~/HW3 — ssh jadhav2@csegrid.ucdenver.pvt — 208x65

[jadhav2@cscs-gnode-01 HW3]$ ./question4_malloc_leak
[jadhav2@cscs-gnode-01 HW3]$ gdb question4_malloc_leak
GNU gdb (GDB) Red Hat Enterprise Linux 12.1-5.el9
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-redhat-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from question4_malloc_leak...
(gdb) run
Starting program: /csegrid/home/jadhav2/HW3/question4_malloc_leak
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib64/libthread_db.so.1".
[Inferior 1 (process 95492) exited normally]
Missing separate debuginfos, use: dnf debuginfo-install glibc-2.34-231.el9_7.10.x86_64
(gdb) exit
```

When the program is run using gdb, no problem is reported. The debugger shows that the program exits normally, because gdb mainly detects runtime crashes or errors, not memory leaks.

Running with valgrind:

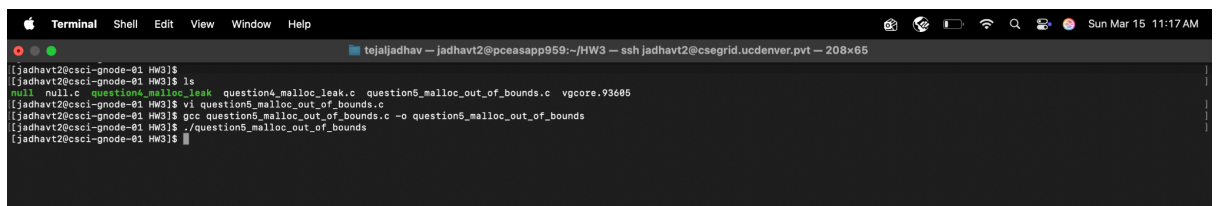
```
[jadhav2@cscs-gnode-01 HW3]$ valgrind --leak-check=yes ./question4_malloc_leak
==96501== Memcheck, a memory error detector
==96501== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==96501== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==96501== Command: ./question4_malloc_leak
==96501==
==96501== HEAP SUMMARY:
==96501==    in use at exit: 400 bytes in 1 blocks
==96501==   total heap usage: 1 allocs, 0 frees, 400 bytes allocated
==96501==
==96501== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==96501==    at 0x484682F: malloc (vg_replace_malloc.c:446)
==96501==    by 0x401137: main (question4_malloc_leak.c:14)
==96501==
==96501== LEAK SUMMARY:
==96501==    definitely lost: 400 bytes in 1 blocks
==96501==    indirectly lost: 0 bytes in 0 blocks
==96501==    possibly lost: 0 bytes in 0 blocks
==96501==    still reachable: 0 bytes in 0 blocks
==96501==    suppressed: 0 bytes in 0 blocks
==96501==
==96501== For lists of detected and suppressed errors, rerun with: -s
==96501== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
[jadhav2@cscs-gnode-01 HW3]$
```

When the program is run using Valgrind with --leak-check=yes, the tool reports that 400 bytes in 1 block are definitely lost. This indicates a memory leak, since memory

was allocated using malloc() but never freed. Valgrind successfully detects this issue and reports it in the output.

5. Write a program that creates an array of integers called data of size 100 using malloc; then, set data[100] to zero. What happens when you run this program? What happens when you run this program using valgrind? Is the program correct?

Output:

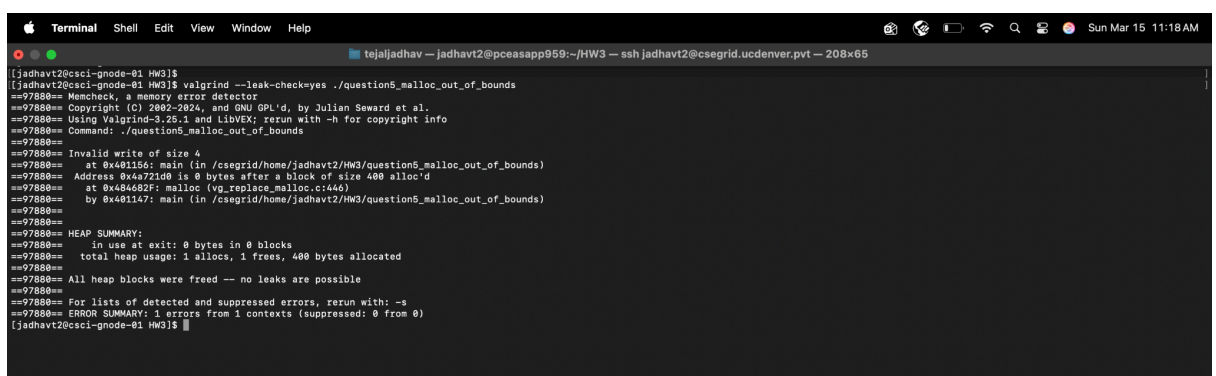


```
tejal@jadhav:~$ cd /home/jadhav/Workspace/valgrind/
tejal@jadhav:~/Workspace/valgrind$ ls
question4_malloc_leak.c  question5_malloc_out_of_bounds.c  vgcore.93605
tejal@jadhav:~/Workspace/valgrind$ gcc question5_malloc_out_of_bounds.c -o question5_malloc_out_of_bounds
tejal@jadhav:~/Workspace/valgrind$ ./question5_malloc_out_of_bounds
```

What happens when you run this program?

When the program runs normally, it may not show any visible error and may appear to work. However, the program writes to data[100], which is outside the allocated array. The valid indices for the array are 0–99, so this causes undefined behavior and may corrupt memory.

Running with valgrind:



```
tejal@jadhav:~$ cd /home/jadhav/Workspace/valgrind/
tejal@jadhav:~/Workspace/valgrind$ valgrind --leak-check=yes ./question5_malloc_out_of_bounds
==7880== Memcheck, a memory error detector
==7880== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==7880== Using Valgrind-3.25.1 and libVEX; rerun with -h for copyright info
==7880== Command: ./question5_malloc_out_of_bounds
==7880== Invalid write of size 4
==7880==    at 0x401156: main (in /csegrid/home/jadhav2/HW3/question5_malloc_out_of_bounds)
==7880== Address 0x4721d0 is 0 bytes after a block of size 400 alloc'd
==7880==    at 0x484682F: malloc (vg_replace_malloc.c:446)
==7880==    by 0x401147: main (in /csegrid/home/jadhav2/HW3/question5_malloc_out_of_bounds)
==7880==
==7880== HEAP SUMMARY:
==7880==   in use at exit: 0 bytes in 0 blocks
==7880==   total heap usage: 1 allocs, 1 frees, 400 bytes allocated
==7880== All heap blocks were freed -- no leaks are possible
==7880== For lists of detected and suppressed errors, rerun with: -s
==7880== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
tejal@jadhav:~/Workspace/valgrind$
```

What happens when you run this program using valgrind?

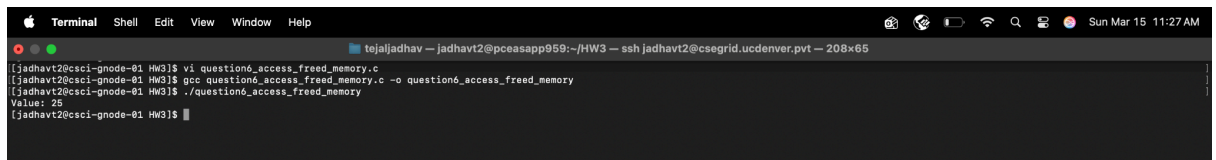
When the program is run using Valgrind with --leak-check=yes, Valgrind reports an “Invalid write of size 4”. It indicates that the program attempted to write 0 bytes after a block of size 400 allocated, meaning the program wrote beyond the allocated memory.

Is the program correct?

No, the program is not correct because it accesses memory outside the allocated array. The last valid index is data[99], but the program attempts to write to data[100], which causes an out-of-bounds memory access.

6. Create a program that allocates an array of integers (as above), frees them, and then tries to print the value of one of the elements of the array. Does the program run? What happens when you use valgrind on it?

Output:

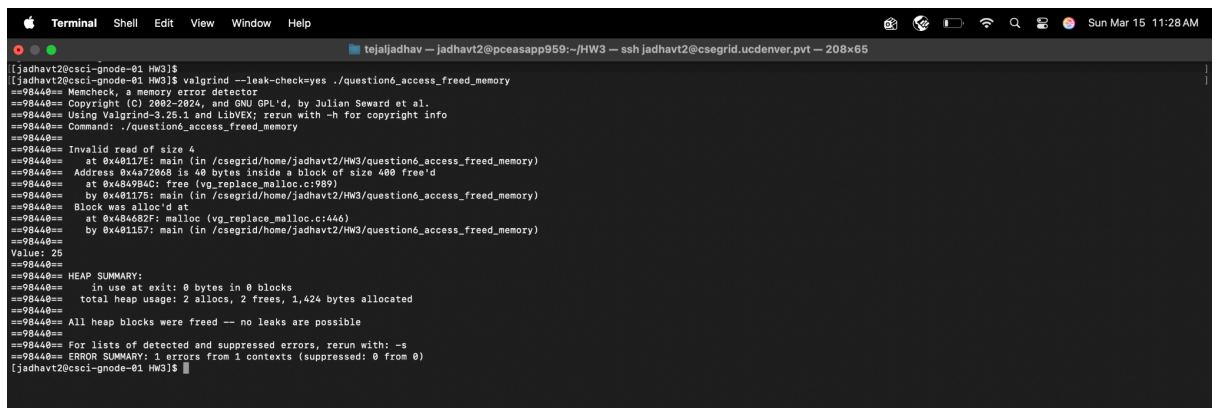


```
tejaljadhav — jadhav2@pceasapp959:~/HW3 — ssh jadhav2@csegrid.ucdenver.pvt — 208x65
[jadhav2@csci-gnode-01 HW3]$ vi question6_access_freed_memory.c
[jadhav2@csci-gnode-01 HW3]$ gcc question6_access_freed_memory.c -o question6_access_freed_memory
[jadhav2@csci-gnode-01 HW3]$ ./question6_access_freed_memory
Value: 25
[jadhav2@csci-gnode-01 HW3]$
```

Does the program run?

Yes, the program runs and prints a value (25) even after the memory is freed. However, this is undefined behavior because the program is accessing memory that has already been freed.

Running with valgrind:



```
tejaljadhav — jadhav2@pceasapp959:~/HW3 — ssh jadhav2@csegrid.ucdenver.pvt — 208x65
[jadhav2@csci-gnode-01 HW3]$
[jadhav2@csci-gnode-01 HW3]$ valgrind --leak-check=yes ./question6_access_freed_memory
==98440== Memcheck, a memory error detector
==98440== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==98440== Using Valgrind-3.25.1 and LibVEX; rerun with -h for copyright info
==98440== Command: ./question6_access_freed_memory
==98440==
==98440== Invalid read of size 4:
==98440==    at 0x40117E: main (in /csegrid/home/jadhav2/HW3/question6_access_freed_memory)
==98440== Address 0x472068 is 40 bytes inside a block of size 400 free'd
==98440==    at 0x404984C: free (vg_replace_malloc.c:989)
==98440==    by 0x401175: main (in /csegrid/home/jadhav2/HW3/question6_access_freed_memory)
==98440== Block was alloc'd at
==98440==    at 0x404062F: malloc (vg_replace_malloc.c:446)
==98440==    by 0x401157: main (in /csegrid/home/jadhav2/HW3/question6_access_freed_memory)
==98440==
Value: 25
==98440==
==98440== HEAP SUMMARY:
==98440==    in use at exit: 0 bytes in 0 blocks
==98440==    total heap usage: 2 allocs, 2 frees, 1,424 bytes allocated
==98440==
==98440== All heap blocks were freed -- no leaks are possible
==98440==
==98440== For lists of detected and suppressed errors, rerun with: -s
==98440== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
[jadhav2@csci-gnode-01 HW3]$
```

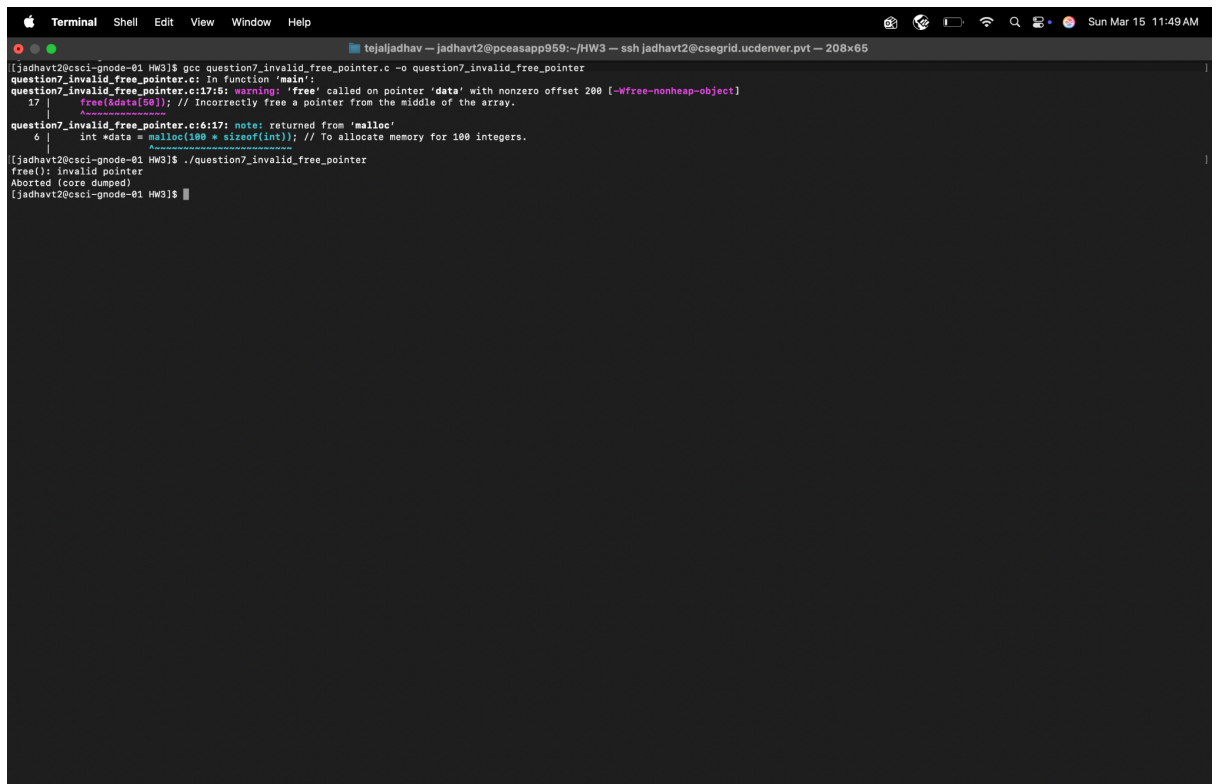
What happens when you run it with Valgrind?

When the program is run with Valgrind, it reports an error: Invalid read of size 4

Valgrind also indicates that the program tried to read memory inside a block of size 400 that had already been freed. This means the program accessed memory after calling free().

7. Now pass a funny value to free (e.g., a pointer in the middle of the array you allocated above). What happens? Do you need tools to find this type of problem?

Output:



```
tejaljadhav — jadhavt2@pceasapp959:~/HW3 — ssh jadhavt2@csegrid.ucdenver.pvt — 208x65
[jadhavt2@csci-gnode-01 HW3]$ gcc question7_invalid_free_pointer.c -o question7_invalid_free_pointer
question7_invalid_free_pointer.c: in function 'main':
question7_invalid_free_pointer.c:17:5: warning: 'free' called on pointer 'data' with nonzero offset 200 [-Wfree-nonheap-object]
   17 |     free(&data[50]); // Incorrectly free a pointer from the middle of the array.
      |     ^
question7_invalid_free_pointer.c:6:17: note: returned from 'malloc'
    6 |     int *data = malloc(100 * sizeof(int)); // To allocate memory for 100 integers.
      |                 ^
[jadhavt2@csci-gnode-01 HW3]$ ./question7_invalid_free_pointer
free(): invalid pointer
Aborted (core dumped)
[jadhavt2@csci-gnode-01 HW3]$
```

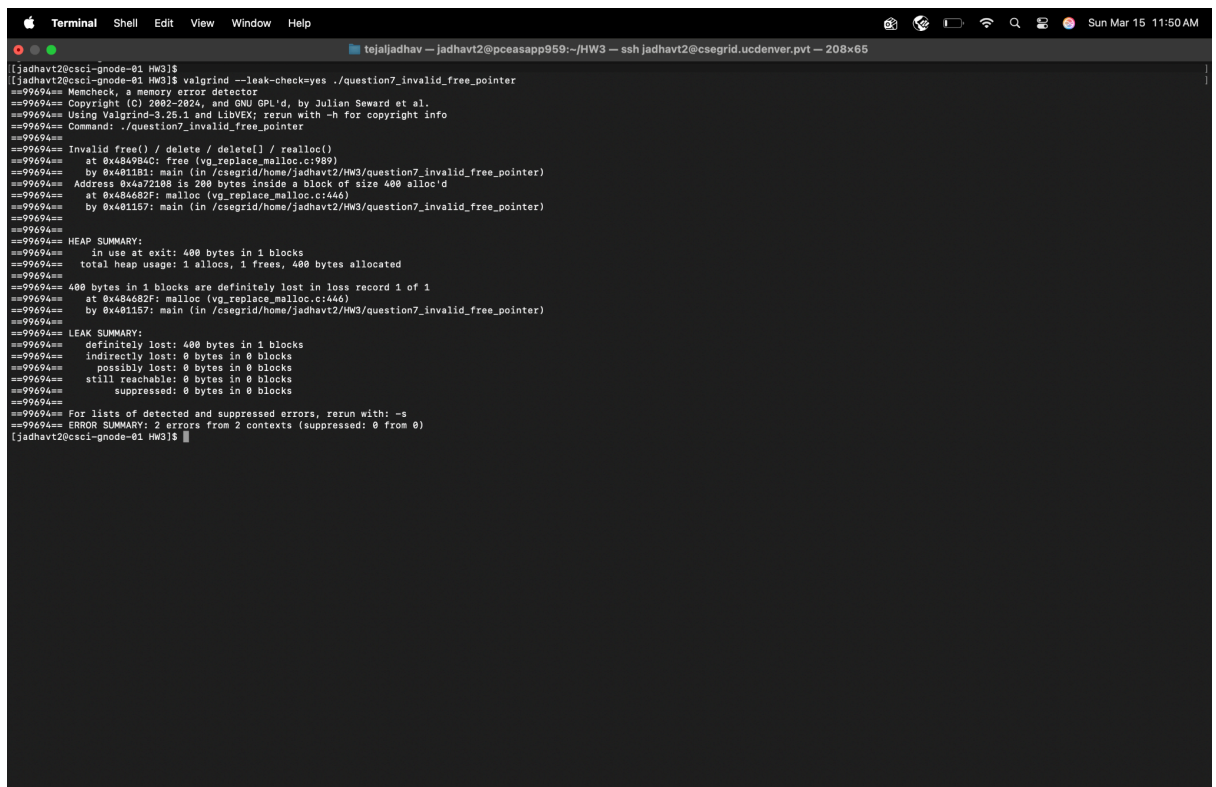
What happens when you run the program?

When the program runs, it crashes and prints:

free(): invalid pointer
Aborted (core dumped)

This happens because free() is called on &data[50], which is a pointer inside the allocated array instead of the original pointer returned by malloc(). The free() function must only be used with the exact pointer returned by malloc().

Running with valgrind:



```
tejaljadhav — jadhvt2@pceasapp959:~/HW3 — ssh jadhvt2@csegrid.ucdenver.pvt — 208x65
[jadhvt2@csci-gnode-01 HW3]$
[jadhvt2@csci-gnode-01 HW3]$ valgrind --leak-check=yes ./question7_invalid_free_pointer
==99694== Memcheck, a memory error detector
==99694== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==99694== Using Valgrind-3.25.1 and libVEX; rerun with -h for copyright info
==99694== Command: ./question7_invalid_free_pointer
==99694==
==99694== Invalid free() / delete / delete[] / realloc()
==99694==    at 0x4849B4C: free (vg_replace_malloc.c:989)
==99694==    by 0x401151: main (in /csegrid/home/jadhvt2/HW3/question7_invalid_free_pointer)
==99694==    Address 0x4772108 is 200 bytes inside a block of size 400 alloc'd
==99694==    at 0x484682F: malloc (vg_replace_malloc.c:446)
==99694==    by 0x401157: main (in /csegrid/home/jadhvt2/HW3/question7_invalid_free_pointer)
==99694==
==99694== HEAP SUMMARY:
==99694==    in use at exit: 400 bytes in 1 blocks
==99694==    total heap usage: 1 allocs, 1 frees, 400 bytes allocated
==99694==
==99694== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==99694==    at 0x484682F: malloc (vg_replace_malloc.c:446)
==99694==    by 0x401157: main (in /csegrid/home/jadhvt2/HW3/question7_invalid_free_pointer)
==99694==
==99694== LEAK SUMMARY:
==99694==    definitely lost: 400 bytes in 1 blocks
==99694==    indirectly lost: 0 bytes in 0 blocks
==99694==    possibly lost: 0 bytes in 0 blocks
==99694==    still reachable: 0 bytes in 0 blocks
==99694==    suppressed: 0 bytes in 0 blocks
==99694==
==99694== For lists of detected and suppressed errors, rerun with: -s
==99694== ERROR SUMMARY: 2 errors from 2 contexts (suppressed: 0 from 0)
[jadhvt2@csci-gnode-01 HW3]$
```

valgrind shows: Invalid free() / delete / delete[] / realloc()

It also shows that the address being freed is 200 bytes inside a block of size 400 allocated, meaning the program attempted to free memory from the middle of the allocated block.

Valgrind also reports a memory leak of 400 bytes, because the original pointer returned by malloc() was never freed correctly.

Do you need tools to find this type of problem?

Yes. Tools like Valgrind are very helpful because they detect invalid free operations and memory leaks, and clearly show where the problem occurs.